Visión por Computadora 3D

Cámaras y Profundidad

MSc. Fernando Silva

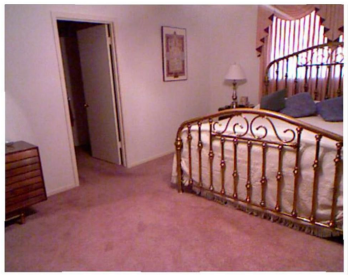


Representaciones 3D

Representaciones 3D

Mapa de profundidad (Depth Map)

Un mapa de profundidad es una imagen en la que cada píxel contiene información sobre la distancia de un punto de la escena respecto a la cámara.



(a) RGB Image



(a) Depth Map



(b) RGB Image



(b) Depth Map

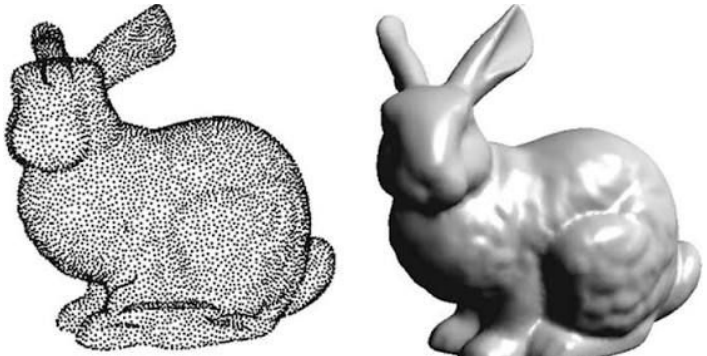
- **Estructura:** matriz bidimensional de valores de profundidad
- **Características:** conserva la perspectiva de la cámara, pero no representa directamente la geometría completa del objeto
- **Ejemplo:** estimar la profundidad de una habitación a partir de una fotografía utilizando Depth Anything
- **Aplicaciones:** conducción autónoma, realidad aumentada, robótica y reconstrucción 3D

Limitaciones: solo contiene información de las superficies visibles desde una perspectiva determinada y algunos modelos estiman profundidad relativa en lugar de distancias métricas.

Representaciones 3D

Nube de puntos (Point Cloud)

Una nube de puntos representa objetos o escenas mediante un conjunto de puntos distribuidos en el espacio tridimensional. También puede almacenar atributos adicionales, como color RGB, intensidad o normales.



Point Cloud Representation

Bunny 3D Model

Cada punto tiene coordenadas:

$$P_i = [X_i, Y_i, Z_i]^T$$

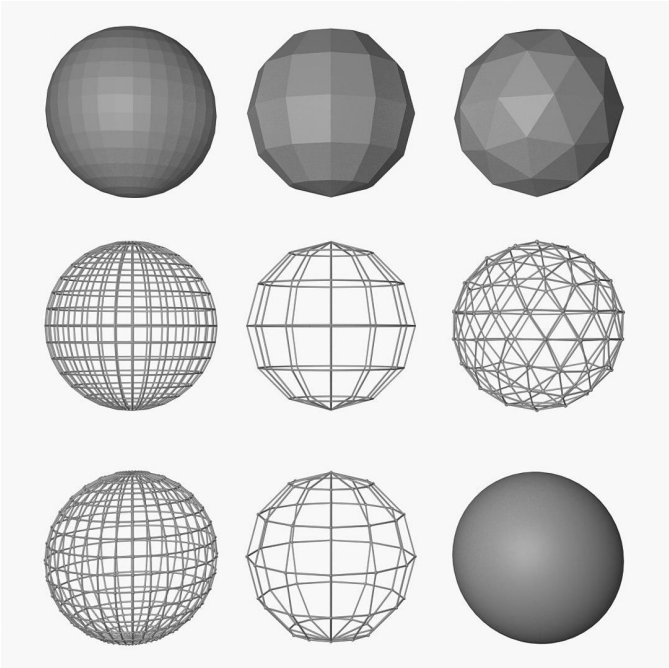
- **Estructura:** conjunto de puntos tridimensionales sin conexiones explícitas
- **Características:** permite representar geometrías complejas obtenidas mediante sensores o algoritmos de reconstrucción
- **Ejemplo:** un escaneo LiDAR de una calle que captura edificios, vehículos y árboles
- **Aplicaciones:** vehículos autónomos, cartografía 3D, robótica y reconstrucción de objetos

Limitación: los puntos no están conectados entre sí por lo que no definen directamente una superficie continua.

Representaciones 3D

Malla 3D (Mesh)

Una malla 3D representa la superficie de un objeto mediante vértices conectados por aristas y caras, generalmente triangulares.



Sus componentes principales son:

Vértices: puntos con coordenadas (X,Y,Z)

Aristas: conexiones entre vértices

Caras: polígonos que forman la superficie del objeto

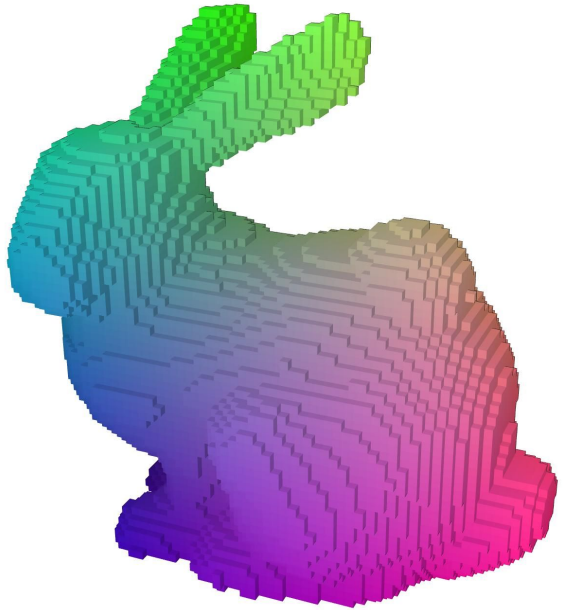
- **Ejemplo:** un modelo 3D de una silla puede estar compuesto por miles de triángulos que describen su geometría
- **Aplicaciones:** videojuegos, animación, impresión 3D, simulaciones físicas y diseño CAD

Limitación: construir mallas de buena calidad a partir de datos incompletos puede ser complejo

Representaciones 3D

Vóxeles (Voxels)

Los vóxeles son el equivalente tridimensional de los píxeles, mientras que una imagen se divide en una cuadrícula de píxeles, un volumen 3D se divide en pequeñas celdas cúbicas llamadas vóxeles.



Cada vóxel puede almacenar información como:

Ocupación: si una región está vacía u ocupada | **Color** | **Densidad** | **Propiedades físicas**

- **Ejemplo:** representar un objeto mediante una cuadrícula de cubos o almacenar un volumen obtenido mediante tomografía computarizada
- **Aplicaciones:** imágenes médicas, robótica, simulaciones volumétricas y reconstrucción 3D

Limitación: una resolución elevada puede requerir mucha memoria, especialmente cuando se utiliza una cuadrícula densa

Todas estas representaciones parten de lo mismo: saber a qué distancia está cada punto... pero la cámara, al sacar la foto, tira justo esa información

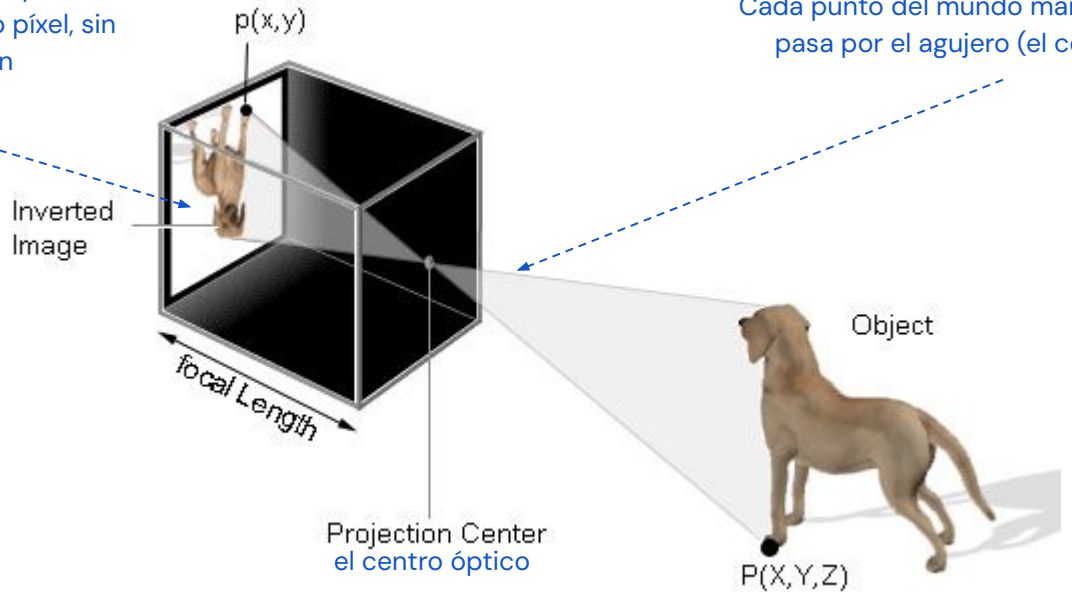
Modelo de Cámara Pinhole

Modelo de Cámara Pinhole

Una cámara convierte un punto 3D en un píxel dividiendo por la profundidad, por eso, desde una sola foto la profundidad se pierde

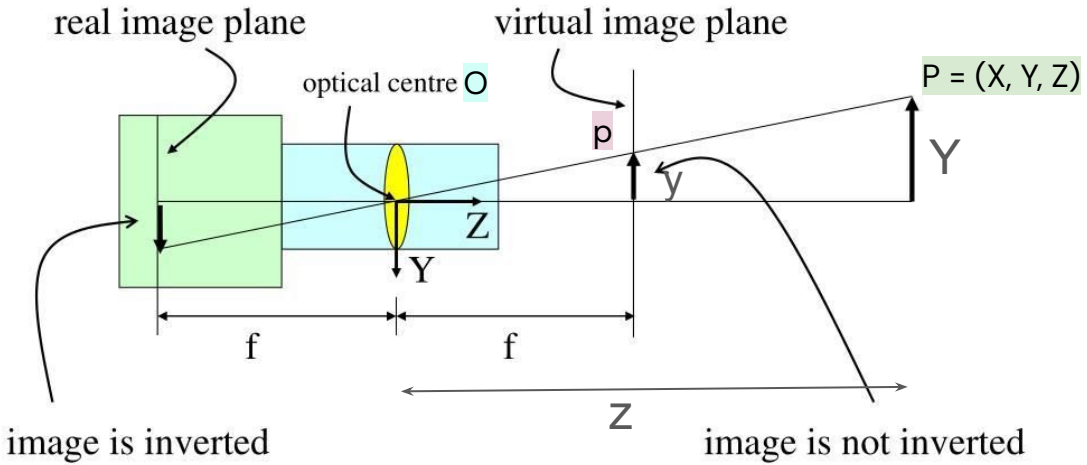
Esto llega al fondo de la caja, todos los puntos sobre el mismo rayo caen en el mismo píxel, sin importar a qué distancia estén

Cada punto del mundo manda un rayo que pasa por el agujero (el centro óptico)



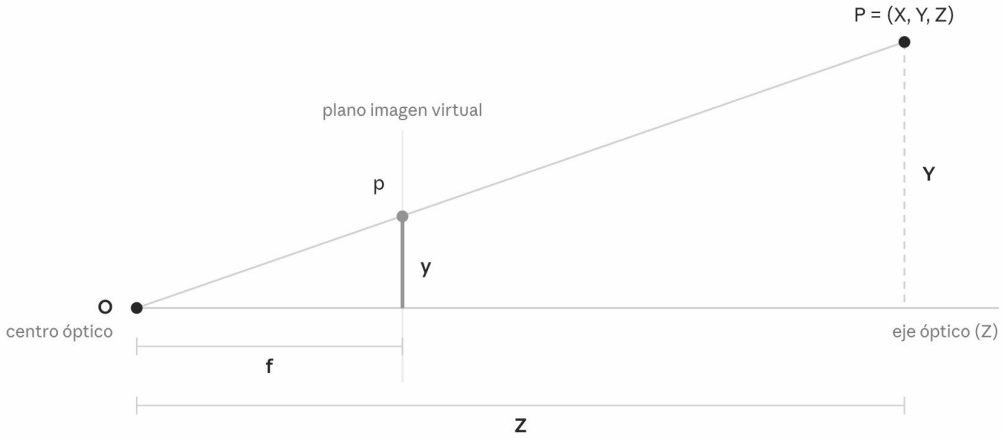
Modelo de Cámara Pinhole

- se toma el centro óptico O como origen del sistema de la cámara y el plano imagen en Z = f
- un punto P = (X, Y, Z) se une con O mediante un rayo, que corta al plano imagen en p = (x, y)



los dos triángulos comparten el ángulo en O, por eso y/f = Y/Z

Modelo de Cámara Pinhole



- el **P** grande tiene catetos Z (a lo largo del eje óptico) e Y (altura del punto)
- el **p** pequeño tiene catetos f e y
- como los ángulos son iguales, los triángulos son semejantes y sus catetos son proporcionales:

  y / f = Y / Z

- el resultado es la ecuación de proyección perspectiva:

$$x = f\,\frac{X}{Z}, \qquad y = f\,\frac{Y}{Z}$$

Se divide por, lo lejano se ve más chico

Coordenadas Homogéneas

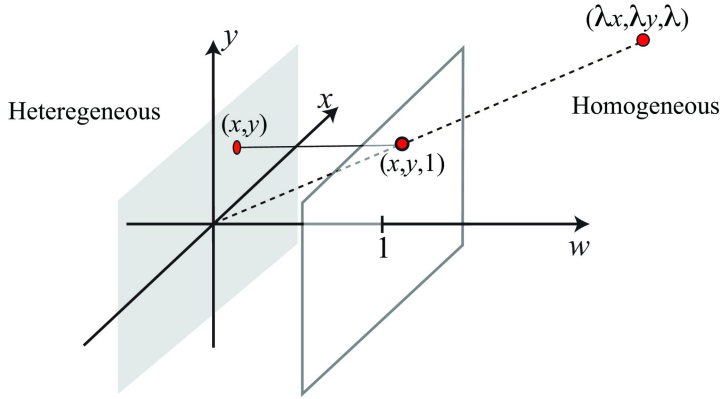
$$x = f\,\frac{X}{Z}, \qquad y = f\,\frac{Y}{Z}$$

- Las divisiones no se pueden escribir como una multiplicación de matrices, las coordenadas homogéneas son un truco para “guardar” esta división y hacerla recién al final
- agregando una coordenada, P = [X, Y, Z, 1]^T, la proyección se escribe como un producto de matrices y al final se divide por la última coordenada (λ):

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Cada fila de la matriz produce un número:

- la primera da f·X, la segunda f·Y y la tercera Z
- el resultado es [f·X, f·Y, Z]^T
- para volver a la imagen, se divide todo por el último número: [f·X / Z, f·Y / Z, 1]^T, que es exactamente la proyección



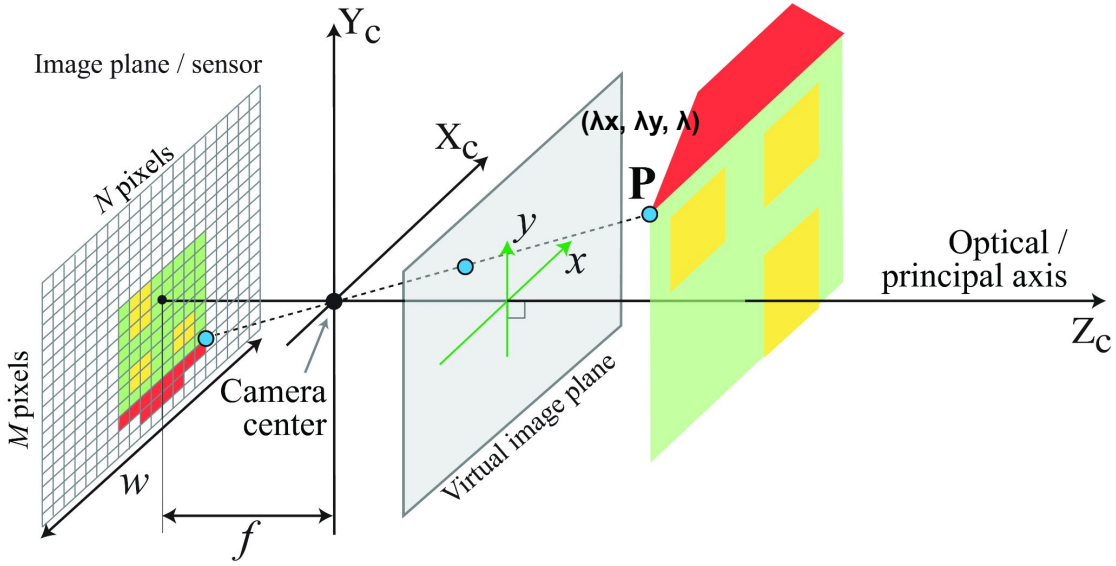
dividir por λ es deslizarse por la recta hasta cortar el plano w = 1

Metros a Píxeles

Esta imagen ilustra el proceso de formación de imágenes y su muestreo mediante el sensor de la cámara. Es necesario considerar la proyección en perspectiva de un punto, expresado en coordenadas de la cámara, hacia las coordenadas de píxeles en el plano del sensor.

$$x = f\,\frac{X}{Z}, \qquad y = f\,\frac{Y}{Z}$$

x e **y** están en metros (en milímetros, en realidad), medidos desde el centro del plano imagen, una foto se guarda en píxeles contados desde la esquina



Una imagen se proyecta sobre el sensor de la cámara. Las coordenadas del mundo se transforman en coordenadas de píxeles en el sensor. La distancia focal es **f**, el ancho físico del sensor es **w** y el sensor tiene **N x M** píxeles.

Metros a Píxeles

- la focal en píxeles es:

la focal en milímetros (al igual que w)

el ancho de la imagen en píxeles

$$a = f \cdot \frac{N}{w}$$

el ancho del sensor es el tamaño físico del chip que captura la luz, medido en milímetros

el alto de la imagen en píxeles

$$b = f \cdot \frac{M}{h}$$

h el alto del sensor

- si el centro óptico no es el píxel $[0, 0]^T$ y los píxeles no son cuadrados, la matriz intrínseca queda:

describe la cámara por dentro y no cambia si la cámara se mueve

$$\mathbf{K} = \begin{bmatrix} a & 0 & c_x \\ 0 & b & c_y \\ 0 & 0 & 1 \end{bmatrix},$$

dicen dónde está el centro de la imagen en píxeles

$$n = a\frac{X}{Z} + c_x, \quad m = b\frac{Y}{Z} + c_y$$

dicen cuántos píxeles ocupa un objeto de 1 m a 1 m de distancia. si hacemos zoom, **a** crece y todo se ve más grande

si conocemos **a, b** y la distancia **Z**, podemos pasar de píxeles a metros

Metros a Píxeles

Ejemplo

Un celular con fotos de:

$$N \times M = 4000 \times 3000 \text{ px}$$

$$a = 3000$$

$$b = 3000$$

$$c_x = 2000$$

$$c_y = 1500$$

$$\mathbf{K} = \begin{bmatrix} a & 0 & c_x \\ 0 & b & c_y \\ 0 & 0 & 1 \end{bmatrix}, \qquad n = a\frac{X}{Z} + c_x, \quad m = b\frac{Y}{Z} + c_y$$

P (m) el punto 3D en coordenadas de la cámara, en metros: [X, Y, Z]	**n (px)** la columna del píxel donde cae ese punto en la foto, contada desde la izquierda	**m (px)** la fila del píxel, contada desde arriba
$[0.5, 0.2, 2]^T$ un punto que está 0.5 m a la derecha del eje óptico, 0.2 m hacia abajo (o arriba, según la convención del eje Y) y a 2 m de distancia hacia adelante	= 3000 × 0.25 + 2000 = 2750	= 3000 × 0.1 + 1500 = 1800
$[0.5, 0.2, 4]^T$	= 3000 × 0.125 + 2000 = 2375	= 3000 × 0.05 + 1500 = 1650

El mismo objeto al doble de distancia queda a la mitad de distancia del centro, este es el efecto que un modelo de profundidad tiene que deshacer

Si una cámara pierde Z, qué pasa si usamos dos?
es lo que hacemos nosotros con los ojos

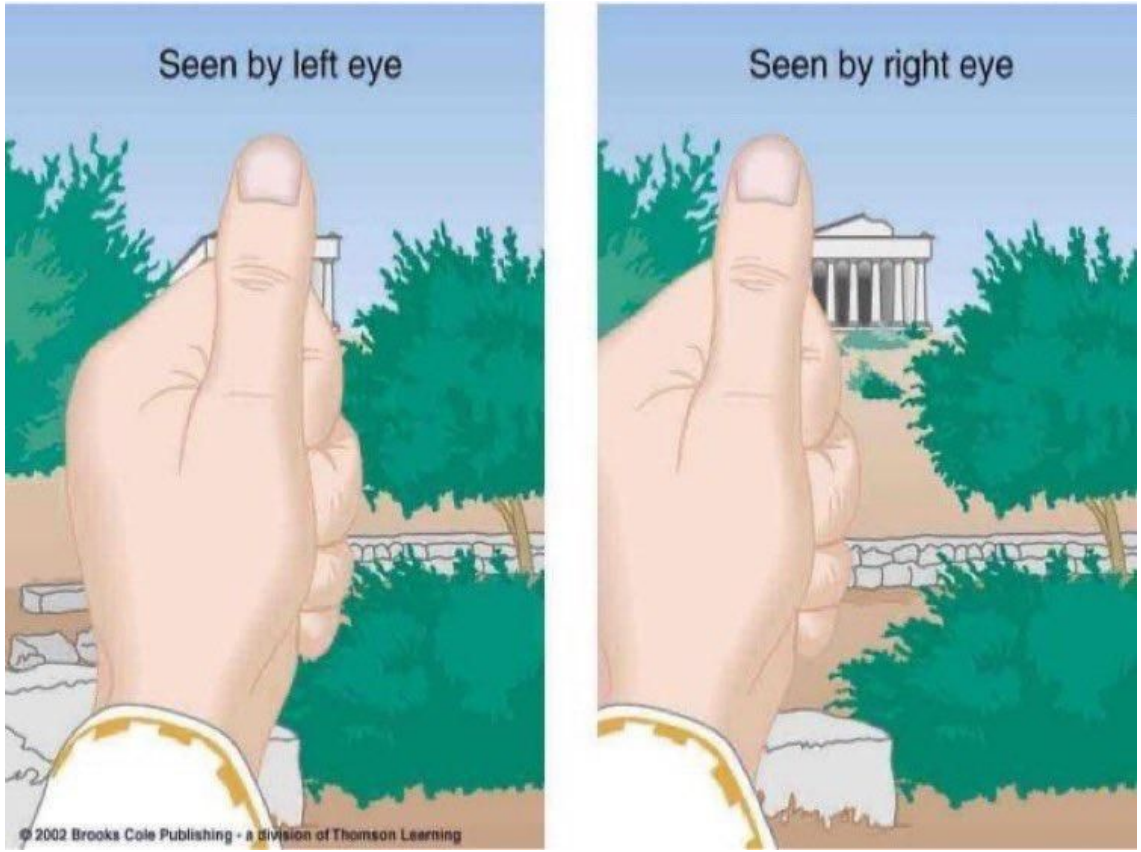
Estéreo y Geometría Epipolar

Estéreo y Geometría Epipolar

Con K conocida cada píxel se convierte en un rayo: sabemos la dirección en la que está el punto. Pero un rayo no dice a qué distancia está: la silla puede estar en cualquier lugar de esa línea, cómo resolvemos esto las personas? con dos cámaras

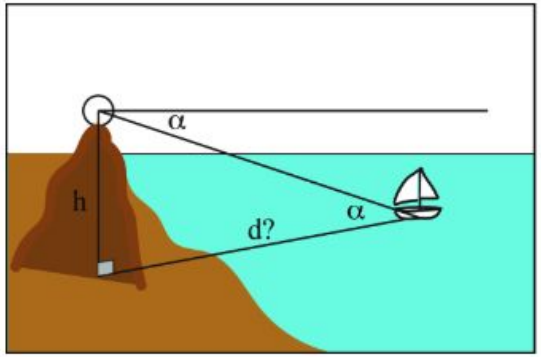
Con dos cámaras separadas, cada punto queda en la intersección de dos rayos y la profundidad sale de la disparidad entre las dos imágenes, el error crece con el cuadrado de la distancia

Estéreo y Geometría Epipolar

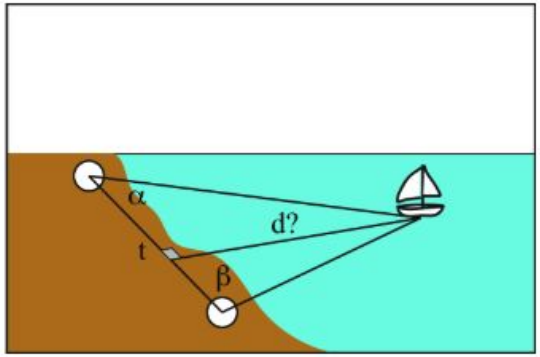


Triangulación

Dos métodos para estimar la distancia de un barco a la costa



El primer método utiliza un único punto de observación y requiere conocer la altura del observador sobre el nivel del agua.



El segundo método utiliza dos puntos de observación

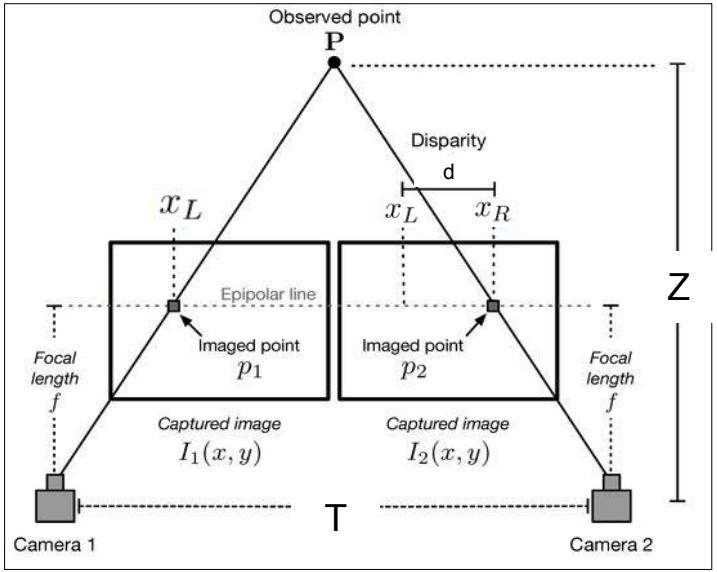
Para saber a qué distancia d está un barco, se mide desde dos puntos de la costa separados por t el ángulo hacia el barco, α y β:

$$d = t\,\frac{\sin(\alpha)\sin(\beta)}{\sin(\alpha+\beta)}$$

Con cámaras la idea es la misma: cada píxel define un rayo y el punto P está donde se cruzan

Estéreo con Cámaras Paralelas

el caso del barco, con las cámaras mirando en la misma dirección



- dos cámaras iguales, separadas una distancia T
- **P** aparece en x_L en la izquierda y en x_R en la derecha, en la misma fila, por triángulos semejantes

Como las cámaras son paralelas, no hacen falta senos: basta con una resta

con coordenadas en píxeles se usa **a** en lugar de **f**

$$Z = \frac{f\,T}{x_L - x_R} = \frac{f\,T}{d}$$

es la disparidad, inversamente proporcional a Z

la fórmula dice lo mismo que el dedo:

- si Z es chico (cerca), d es grande (salta mucho)
- si Z es grande (lejos), d es chico
- T es la distancia entre los ojos o las cámaras, cuanto más separadas más salta todo y más fácil es medir

El estéreo da metros porque conocemos T… pero tiene dos problemas: no siempre encuentra la pareja de cada píxel y lejos el error crece rápido, y si medimos la distancia directamente?

Sensores de Profundidad

Sensores de Profundidad

- con dos cámaras calibradas, la profundidad sale de medir cuánto se movió cada punto

- pero hay dos problemas prácticos: encontrar la pareja de cada píxel no es fácil (paredes lisas, reflejos), y lejos el error crece rápido

Z de cada píxel, guardado en la posición (n, m)

- cada sensor entrega un mapa **z[n, m]** con un principio físico distinto y falla en situaciones distintas, la profundidad aprendida aparece justo donde los sensores son caros o no alcanzan

- ¿y si en lugar de buscar parejas, medimos la distancia directamente?

Sensores de Profundidad

Sensor	Cómo mide	Alcance aproximado	Falla con	Ejemplo
Estéreo pasivo	Disparidad d entre dos cámaras	Depende de T; de 1 a 40 m en autos	Superficies sin textura, poca luz	Cámaras ZED, estéreo de autos
Luz estructurada	Proyecta un patrón infrarrojo y mide su deformación	Hasta unos 4 m	Luz solar directa	Kinect v1, desbloqueo facial
ToF	Tiempo que tarda la luz en ir y volver	De 0.5 a 5 m en interiores	Reflejos múltiples, superficies muy oscuras	Sensor LiDAR del iPhone, Azure Kinect
LiDAR de escaneo	Pulsos láser que barren la escena	Decenas a más de 100 m	Lluvia, niebla, superficies negras o espejadas; costo	Autos autónomos, topografía
Monocular aprendida	Una red h_θ estima ẑ desde una foto I	Cualquiera, con escala incierta	Escenas fuera de lo que vio en entrenamiento	Depth Anything 3

los sensores miden bien, pero son caros, limitados o no están disponibles, ¿podemos sacar profundidad de una sola cámara común?

Profundidad Relativa vs Métrica

Profundidad Relativa vs Métrica

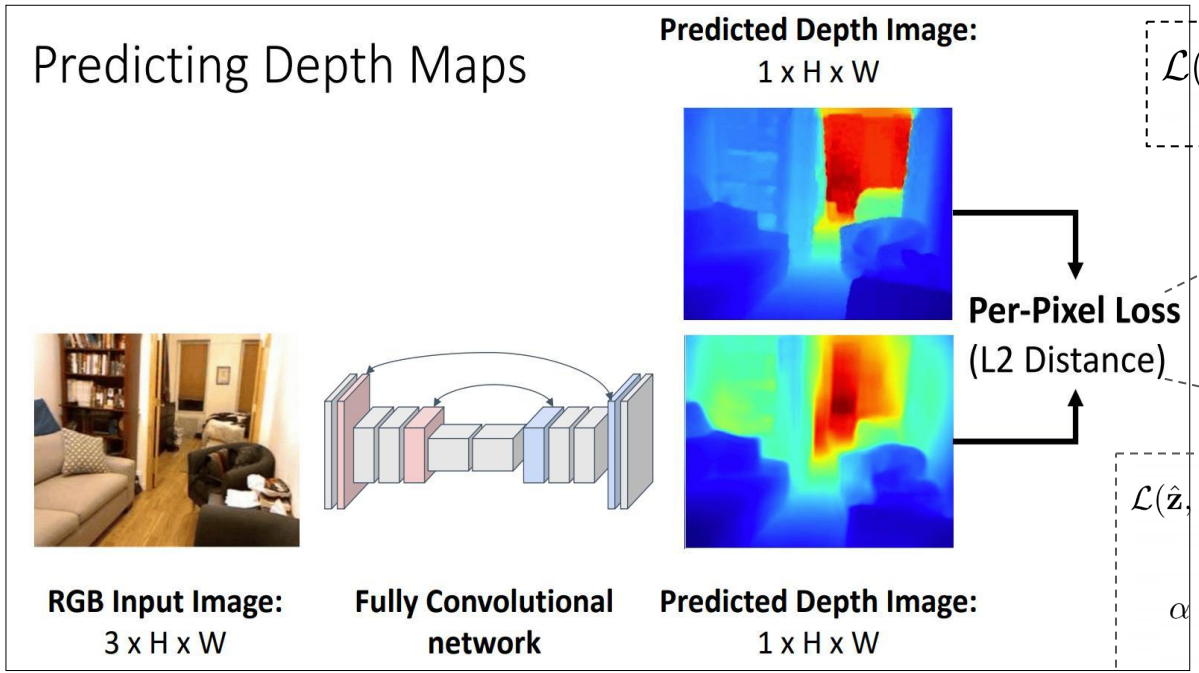
Si cerramos un ojo no dejamos de estimar distancias porque usamos otras pistas: el tamaño conocido de las cosas, qué tapa a qué, las sombras, cómo se achican las baldosas hacia el fondo, etc... nadie nos enseñó reglas para eso ya que lo aprendimos viendo el mundo real.

Formalizando: desde una sola **imagen I**, una red puede saber qué está más cerca que qué



Profundidad Relativa vs Métrica

Idea



L2: supone que la profundidad real se puede estimar, cosa que la ambigüedad de escala impide

$$\mathcal{L}(\hat{\mathbf{z}}, \mathbf{z}) = \sum_{n,m} (\hat{z}[n,m] - z[n,m])^2$$

$$\mathcal{L}(\hat{\mathbf{z}}, \mathbf{z}) = \sum_{n,m} (\log \hat{z}[n,m] - \log z[n,m] - \alpha)^2$$

$$\alpha = \frac{1}{NM} \sum_{n,m} (\log \hat{z}[n,m] - \log z[n,m])$$

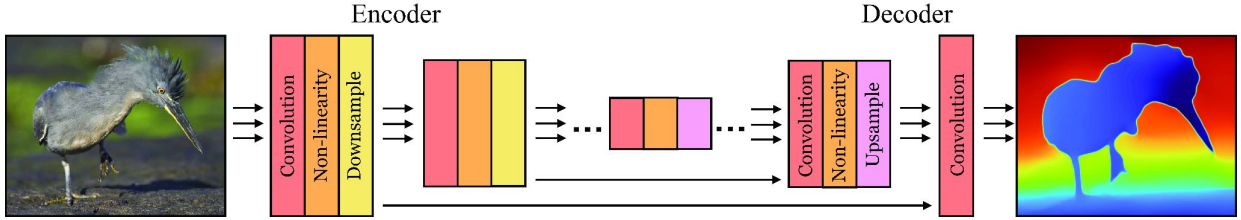
Pérdida invariante a escala: elimina el factor global restando la diferencia media α. Se aplica sobre log z, para que el error en puntos lejanos no domine

Ya sabemos qué le falta a una red para dar metros: la escala y la cámara, ahora veremos cómo los modelos actuales resuelven cada pieza

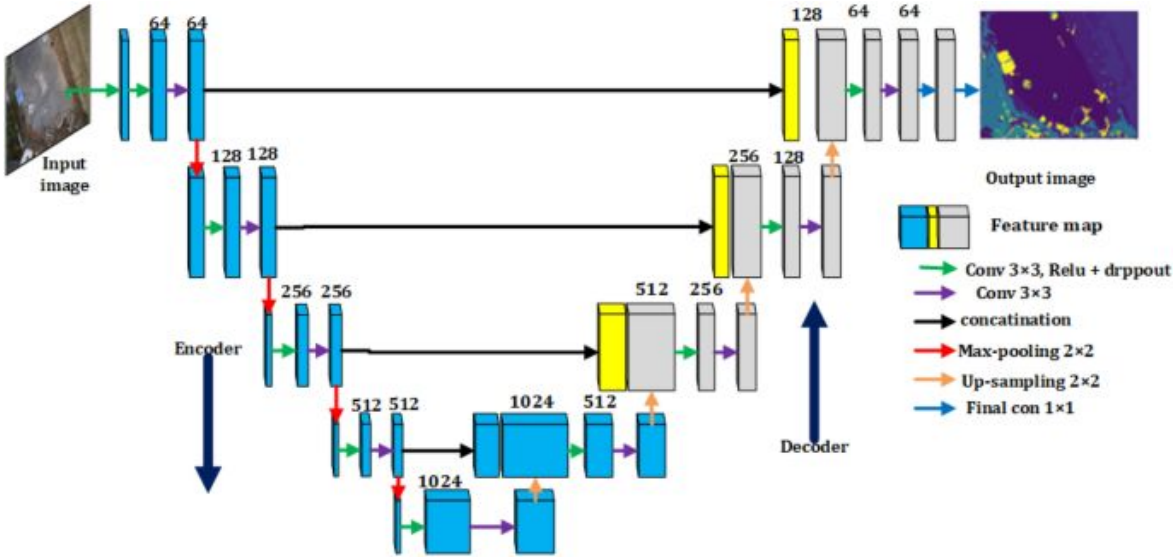
Modelos Actuales y Estéreo Aprendido

Modelos Actuales y Estéreo Aprendido

Podemos estimar la profundidad a través de arquitecturas encoder-decoder (U-Net):



En vez de obtener la image segmentation [1,H,W] vamos a obtener el depth map [1, H, W]



Modelos Actuales y Estéreo Aprendido

U-Net CNN



LINK

Modelos Actuales y Estéreo Aprendido

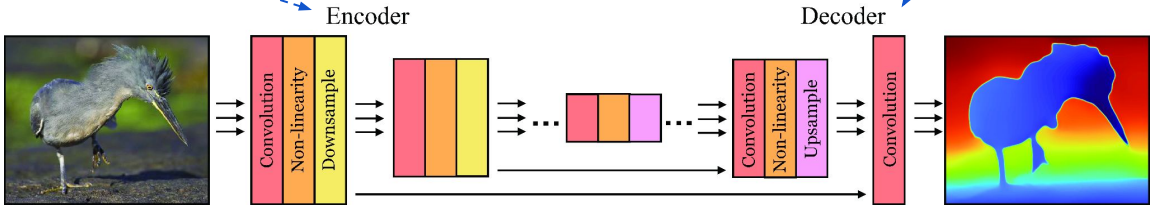
casi todos los modelos actuales implementan h_θ con la misma estructura:

- un encoder Transformer preentrenado
- un decoder que arma un mapa denso
- Muchos datos

lo que cambia es el tipo de salida y cómo resuelven la escala

un ViT preentrenado con auto-supervisión (DINOv2), que ya "entiende" bordes, objetos y perspectiva

(tipo DPT) toma features de varias capas del encoder y las combina para recuperar la resolución completa



los datos son una mezcla de datasets reales, sintéticos (z perfecta) y pseudo-etiquetas generadas por un modelo maestro

para convertir una foto en 3D necesitamos dos cosas por cada píxel: a qué distancia está (z, en metros) y hacia dónde apunta (el rayo, que sale de K), desde una sola foto no conocemos ningun canaal, cada modelo resuelve una parte más de ese problema

Modelos Actuales y Estéreo Aprendido

U-Net Transformers



LINK

Depth Anything V2 (2024)

- Un modelo maestro se entrena solo con 595 mil imágenes sintéticas, que tienen bordes perfectos
- luego etiqueta 62 millones de imágenes reales para entrenar a los modelos alumnos
- salida relativa en disparidad, con versiones métricas ajustadas para interiores y exteriores

Depth Anything V2 predice una disparidad relativa đ (proporcional a 1/z). Sabe qué está más cerca que qué, pero no cuánto mide nada:

$$z = \frac{1}{s\,\hat{d} + o}$$

Faltan tres cosas:

- la escala **s**
- el desplazamiento **o**
- la cámara **K**

Depth Anything V2 (2024)

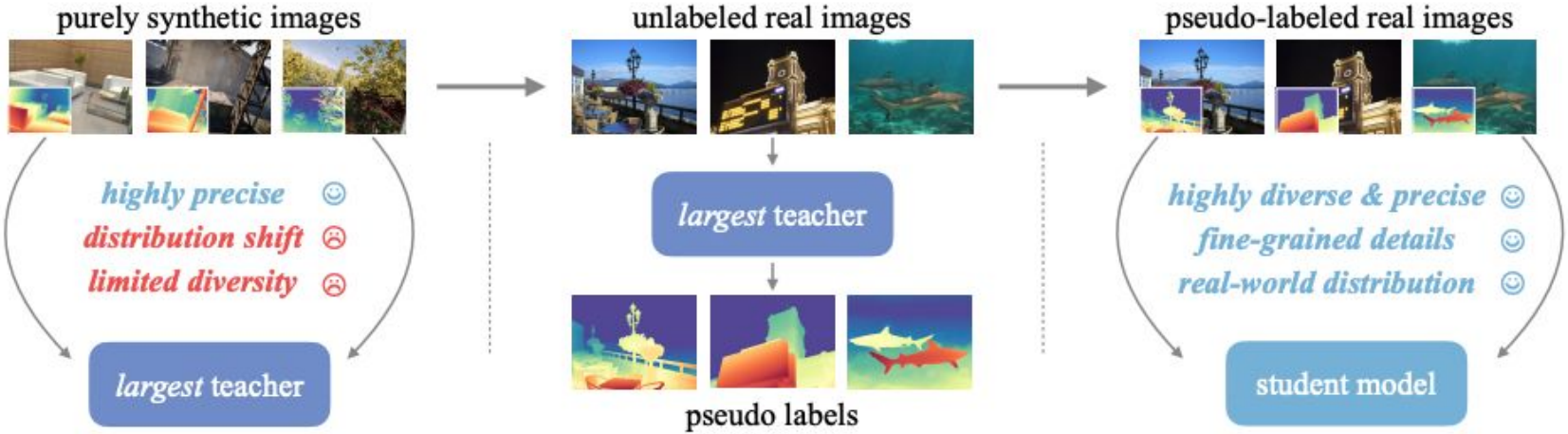
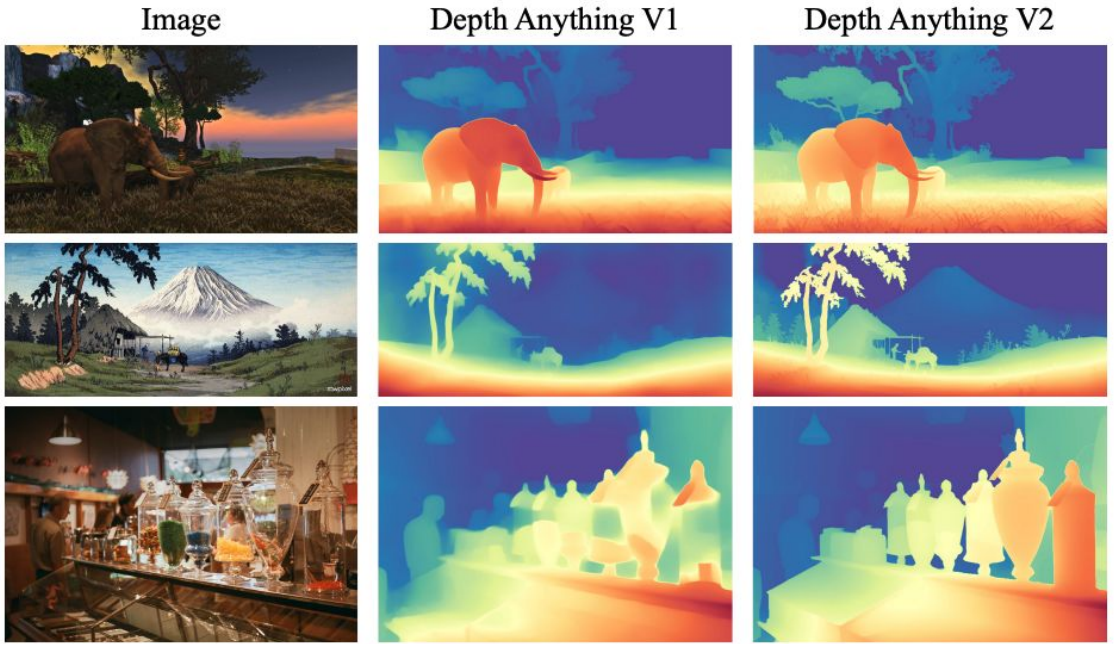


Figure 7: Depth Anything V2. We first train the most capable teacher on precise synthetic images. Then, to mitigate the distribution shift and limited diversity of synthetic data, we annotate unlabeled real images with the teacher. Finally, we train student models on high-quality pseudo-labeled images.

el maestro aprende de lo sintético "corrige la tarea" de millones de fotos reales sin etiquetar y el alumno aprende de esas correcciones, así el alumno ve el mundo real sin que nadie haya medido nada con un sensor

Depth Anything V2 (2024)

- Depth Anything V1: estima la profundidad de una imagen RGB y genera un mapa de profundidad **usando datos reales etiquetados y millones de imágenes sin etiquetar**.
- Depth Anything V2: hace lo mismo, pero genera mapas más precisos, con mejores detalles en los bordes y objetos pequeños, **usando imágenes sintéticas de alta calidad y un proceso de entrenamiento mejorado**.



Depth Anything V3 (2025)

- un Transformer plano (DINO)
- una representación de profundidad + rayo permiten que un mismo modelo haga profundidad monocular, profundidad multivista y estimación de K, R y T



Depth Anything V3 (2025)

usa un único Transformer DINOv2 sin modificaciones que recibe una o varias imágenes y alterna atención dentro de cada imagen con atención entre imágenes

un decoder doble (DualDPT) produce para cada píxel una profundidad y un rayo, y con ambos se obtienen el punto 3D y la cámara

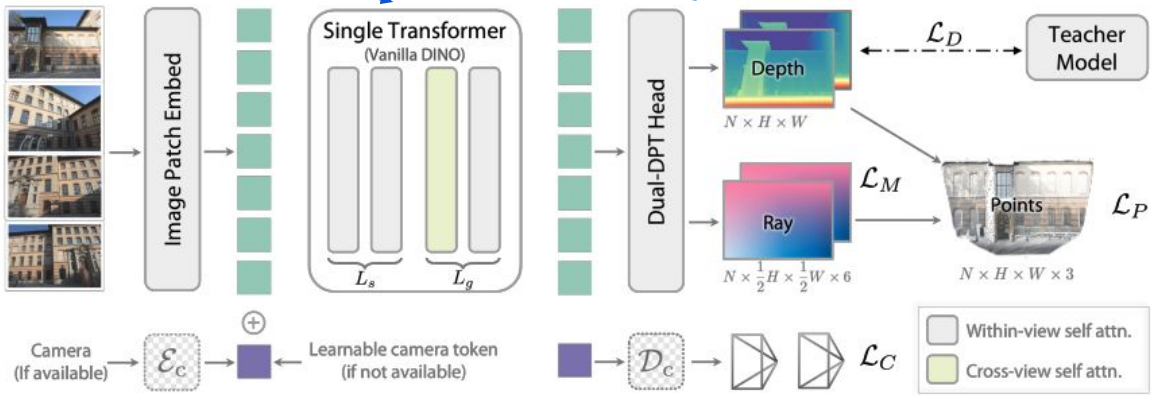


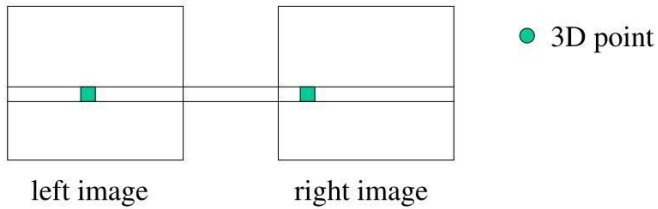
Figure 2 Pipeline of Depth Anything 3. Depth Anything 3 employs a single transformer (vanilla DINOv2 model) without any architectural modifications. To enable cross-view reasoning, an input-adaptive cross-view self-attention mechanism is introduced. A dual-DPT head is used to predict depth and ray maps from visual tokens. Camera parameters, if available, are encoded as camera tokens and concatenated with patch tokens, participating in all attention operations.

Estéreo Aprendido

en estéreo tenemos dos fotos tomadas al mismo tiempo por dos cámaras separadas

- si las imágenes están rectificadas, un punto que aparece en la fila 100 de la imagen izquierda aparece también en la fila 100 de la derecha, solo que corrido hacia un costado
- ese corrimiento es la disparidad d, y con ella sale la profundidad: $Z = a \cdot T / d$

el problema es encontrar, para cada píxel de la izquierda, cuál es su pareja en la derecha, los métodos clásicos comparan colores, pero fallan en paredes lisas, reflejos o zonas con poca luz, donde muchos píxeles se parecen entre sí



Disparity: the difference in image location of the same 3D point when projected under perspective to two different cameras.

$$d = x_{left} - x_{right}$$

Volumen de Costo

- **extraer features:** una red convierte cada píxel de ambas imágenes en un vector que describe su entorno (bordes, texturas, forma), comparar estos vectores es mucho más confiable que comparar colores

- **probar cada disparidad candidata:** para un píxel de la izquierda se toma su vector y se compara con el del píxel que está **d** posiciones más a la izquierda en la imagen derecha, para d = 0, 1, 2... hasta un máximo (por ejemplo, 192), cada comparación da un número: el costo, que es bajo si se parecen y alto si no

- **Apilar todo:** si la imagen tiene H filas y W columnas y probamos D disparidades, guardamos H × W × D costos, ese bloque es el volumen de costo

RAFT-Stereo

Procesar el volumen completo con una red 3D consume mucha memoria y tiempo. RAFT-Stereo lo resuelve así:

- Empieza suponiendo d = 0 en todos los píxeles
- Mira en el volumen de costo solo alrededor de la estimación actual
- Una red recurrente propone una corrección pequeña: “súmale 2”, “réstale 0.5”
- Repite los pasos 2 y 3 varias veces, y la estimación converge

RAFT-Stereo

RAFT-Stereo extrae features de las dos fotos, compara cada píxel de la izquierda con los de su misma fila en la derecha y guarda esas comparaciones en una pirámide de correlación

después arranca con disparidad cero y, en varias iteraciones, una red recurrente consulta esa pirámide y el contexto de la imagen para sumarle pequeñas correcciones hasta llegar al mapa de disparidad final

